

# Retry, DLT & Controlled Replay

Separate business rejection from technical failure • Recover deliberately

## SAY IT IN THE INTERVIEW

“For a retryable processing failure, I allow the initial attempt and two retries with a two-second fixed backoff. If processing still fails, the error handler publishes the record to the DLT. I inspect the failed record, fix the cause and use a targeted replay proof-of-concept.”

## 01 WHICH PATH SHOULD THE RECORD TAKE?

- **Business rejection:** insufficient stock produces `inventory.reservation.failed` and commits normally. Retrying unchanged stock is not our business policy.
- **Technical failure:** an exception leaves the normal ACK path. `DefaultErrorHandler` uses `FixedBackOff(2000L, 2L)`. This is the configured retryable path; do not claim every exception always gets three attempts.

## 02 DLT RECOVERY HAS TO SUCCEED

- **Dead Letter Topic:** a separate topic for records that cannot be processed on the normal path. Here: **`orders.created.avro-dlt`**.
- `DeadLetterPublishingRecoverer` selects the original partition number. Provision the DLT with enough partitions for that routing.
- **`failIfSendResultIsError=true`** surfaces DLT send failure. **`commitRecovered=true`** with `MANUAL_IMMEDIATE` allows the recovered offset to advance after successful recovery.

## 03 INSPECTION IS DIFFERENT FROM REPLAY

- Inspection group: **`inventory-dlt-inspection-group`**. It receives raw bytes, tries Avro decoding and logs the record.
- The inspector ACKs even after a decode failure or null value. This advances its own inspection position; it neither deletes the record nor repairs business data.
- Before replay: identify `eventId/orderId`, inspect and fix the cause. Preserve event identity for the duplicate protection in Chapter 04.

## 04 WHAT THE CURRENT REPLAY RUNNER REALLY DOES

- Enabled only by **`app.dlt.replay.enabled=true`**; disabled by default. Scans DLT partition **1** from the beginning, matching one hard-coded `orderId + eventId`.
- A 30-second scan finds the target; its send waits up to 30 seconds. It republishes to the source topic, then returns from the runner.
- No general replay API, durable replay audit or rate control. Legacy scripts use `orders.created-dlt`, not the current Avro route.

**YAAD RAKHO: reject = business result; DLT = investigate; replay = controlled retry.**

SOURCE CHECK: `KafkaErrorHandlingConfig` + `InventoryDltConsumer` + `InventoryDltReplayRunner`